Reviewer Pack — Minimal Rank of Quantum Logarithm over Tseitin Clauses vs Re…

Entry 7c11c0ce2104 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-25 18:50:42 UTC

Minimal Rank of Quantum Logarithm over Tseitin Clauses vs Resolution Refutation Length

Entry ID: `7c11c0ce2104` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-25 18:50:42 UTC

1. Conjecture

Field A (mathematical branch): Quantum Information Theory (Quantum Logarithm) **Field B** (complexity object): Complexity Theory: Resolution Proof Complexity

Statement:

> ['For a given Tseitin clause set C with n variables, the minimal rank of the quantum logarithm Q(C) is at least Ω(2^(n/4)) and grows exponentially with the resolution refutation length t*(C).', 'Equivalently, for all ε > 0, there exists a constant c such that for every Tseitin clause set C with n variables, if t*(C) = O(2^(n/4 + ε)), then Q(C) has rank at least Ω(2^n/c).']

Rationale (proposer's reasoning):

> ['Quantum logarithms have recently emerged as a powerful tool in quantum information theory and are known to encode complex structures that are difficult to describe classically.', 'This conjecture posits that the complexity of describing these structures using quantum logarithms scales with the resolution refutation length, which would provide new insights into the inherent difficulty of SAT.']

Taxonomy category: `GCT_DET_PERM` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `43a44468f5eebc8c`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, for all ε > 0, at least 80% of the generated Tseitin clause sets with n variables have a quantum logarithm rank Q(C) that meets the condition: Q(C) >= Ω(2^(n/4 + ε)) AND Q(C) <= Ω(2^n/c), where c is a constant.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 7 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `quantum logarithm AND Tseitin clauses resolution proof complexity` - `minimal rank quantum logarithm Tseitin clauses resolution refutation length` - `exponential growth quantum logarithm rank resolution refutation`

Top relevant hits considered: - [http://arxiv.org/abs/2504.12989v3] Query Complexity of Classical and Quantum Channel Discrimination - [http://arxiv.org/abs/2512.10504v2] Tianyan: Cloud services with quantum advantage - [http://arxiv.org/abs/quant-ph/0405018v2] Improved Bounds on the Randomized and Quantum Complexity of Initial-Value Problems - [http://arxiv.org/abs/1405.2700v1] Zero Excess and Minimal Length in Finite Coxeter Groups - [http://arxiv.org/abs/2103.09609v1] Characterizing Tseitin-formulas with short regular resolution refutations - [http://arxiv.org/abs/1709.05771v3] Variational formulas, Busemann functions, and fluctuation exponents for the corner growth model with exponential weights - [http://arxiv.org/abs/1811.10090v1] Exponential Separation between Quantum Communication and Logarithm of Approximate Rank

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=1.8s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(matrix):
        n = len(matrix)
        for i in range(n):
            # Find the pivot
            max_row = i
            for j in range(i+1, n):
                if abs(matrix[j][i]) > abs(matrix[max_row][i]):
                    max_row = j
            matrix[i], matrix[max_row] = matrix[max_row], matrix[i]

            # Eliminate below the pivot
            for j in range(i+1, n):
                factor = matrix[j][i] / matrix[i][i]
                for k in range(n):
```

```
                matrix[j][k] -= factor * matrix[i][k]

    # Back substitution
    solution = [0] * n
    for i in range(n-1, -1, -1):
        solution[i] = matrix[i][-1] / matrix[i][i]
        for j in range(i-1, -1, -1):
            matrix[j][-1] -= matrix[j][i] * solution[i]

    return solution

def rank(matrix):
    n = len(matrix)
    rref_matrix = [row[:] for row in matrix]
    gaussian_elimination(rref_matrix)
    rank = 0
    for row in rref_matrix:
        if any(row):
            rank += 1
    return rank

def tseitin_formula(n):
    variables = list(range(1, n+1))
    clauses = []
    for i in range(1, n+1):
        clauses.append([i])
    for i in range(1, n):
        for j in range(i+1, n+1):
            clauses.append([-i, -j, i+j])
    return variables, clauses

def resolution(clauses):
    new_clauses = set(clauses)
    while True:
        new_clause = None
        for clause1 in new_clauses:
            for clause2 in new_clauses:
                if len(set(clause1) & set(clause2)) == 1:
                    new_clause = [x for x in clause1 + clause2 if x not in set(clause1) & set(
                    break
            if new_clause:
                break
        if new_clause is None:
            return len(new_clauses)
        new_clauses.add(tuple(sorted(new_clause)))

n_values = [5, 10, 15, 20, 30, 40]
results = []
for n in n_values:
    variables, clauses = tseitin_formula(n)
    refutation_length = resolution(clauses)
    q_log_rank = rank([[random.randint(0, 1) for _ in range(n)] for _ in range(n)])
    results.append({
        "n": n,
        "refutation_length": refutation_length,
        "q_log_rank": q_log_rank
```

```
        })

    total_q_log_rank = sum(result["q_log_rank"] for result in results)
    mean_q_log_rank = total_q_log_rank / len(results)
    std_q_log_rank = math.sqrt(sum((result["q_log_rank"] - mean_q_log_rank) ** 2 for result in res

    conjecture_holds = all(result["q_log_rank"] >= 2**(n/4) for n, _, _ in results)
    counterexample = "" if conjecture_holds else "mapping_undefined"

    return {
        "metric_name": "Quantum Logarithm Rank",
        "metric_value": mean_q_log_rank,
        "instances_tested": len(results),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] if sys.argv[1:] else [2**i + 3 for i in range(5, 8)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    total_q_log_rank = sum(result["metric_value"] for result in results)
    mean_q_log_rank = total_q_log_rank / len(results)
    std_q_log_rank = math.sqrt(sum((result["metric_value"] - mean_q_log_rank) ** 2 for result in r

    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_q_log_rank} std={std_q_log_rank} support_fraction={su
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_fai
    else:
        print(f"RESULT: INCONCLUSIVE reason=insufficient_evidence")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5198e0de.py", line 114, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5198e0de.py", line 85, in run_trial
    refutation_length = resolution(clauses)
                        ^^^^^^^^^^^^^^^^^^^
```

```
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5198e0de.py", line 67, in resolution
    new_clauses = set(clauses)
                  ^^^^^^^^^^^^
TypeError: unhashable type: 'list'
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test crashed before producing data, which means that the conjecture could not be evaluated according to the pre-registered support condition. | next: Re-run the test without errors to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11675
2	preregistration	ollama_remote	glm4:latest	0	0	6046
3	novelty	ollama_remote	glm4:latest	0	0	4557
4	novelty	ollama_remote	glm4:latest	0	0	5655
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	37805
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13800
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12116
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	31266
9	judge	ollama_remote	glm4:latest	0	0	60389

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 183310 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/7c11c0ce2104.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/7c11c0ce2104.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/7c11c0ce2104.tar.gz` (if generated)